

UNIVERSIDAD DIEGO PORTALES



FACULTAD DE INGENIERÍA Y CIENCIAS

TAREA 2: ANÁLISIS DEL PROTOCOLO HTTP MEDIANTE CONTENEDORES DOCKER

Estudiantes:

Nicolás Esteban González Fontecilla

David Benjamin Fuentes Castro

Introducción

El desarrollo técnico de las telecomunicaciones modernas descansa fundamentalmente sobre la robustez de las arquitecturas de red y la correcta implementación de sus protocolos de comunicación. Dentro de la capa de aplicación, el Protocolo de Transferencia de Hipertexto (HTTP) representa el pilar fundamental para el intercambio de información distribuida e hipermedia en la red global. Este informe técnico aborda el análisis práctico y empírico de dicho protocolo, empleando un entorno controlado y aislado para observar el comportamiento dinámico de los flujos de datos.

El propósito central de esta actividad es examinar los patrones de comportamiento de HTTP mediante una dupla de software heterogénea bajo el modelo cliente-servidor, utilizando Nginx como demonio servidor y GNU Wget como la herramienta cliente encargada de consumir el recurso. Con el fin de mitigar interferencias externas provenientes del sistema operativo anfitrión y garantizar la reproducibilidad del experimento, la infraestructura se despliega mediante técnicas de contenedorización de procesos sobre una red virtual dedicada.

A lo largo de este documento, se desglosará en primer lugar el sustento teórico subyacente al protocolo HTTP y la configuración explícita de los entornos virtualizados en un sistema operativo basado en Arch Linux. Posteriormente, se presentarán las capturas analíticas obtenidas a través del analizador de paquetes Wireshark, discutiendo minuciosamente los encabezados y códigos de estado observados en las transacciones de datos. Finalmente, se expondrán diversas hipótesis y análisis deductivos relacionados con la inyección o alteración de tráfico en tiempo real y su impacto en la estabilidad de las aplicaciones involucradas.

Desarrollo

Conceptos Teóricos y Tecnologías Utilizadas

El protocolo HTTP es un protocolo de la capa de aplicación (Capa 7 del modelo de referencia OSI) caracterizado por su naturaleza sin estado (*stateless*), lo que implica que cada transacción ejecutada es tratada de forma independiente, careciendo de memoria intrínseca respecto a peticiones previas. HTTP basa su comunicación en un paradigma síncrono de petición-respuesta (*request-response*), interactuando de forma directa sobre la capa de transporte mediante el protocolo orientado a la conexión TCP, comúnmente a través del puerto bien conocido 80 para sus variantes no cifradas.

La arquitectura de contenedores, gestionada a través de Docker, permite instanciar espacios de nombres (*namespaces*) y grupos de control (*cgroups*) aislados del núcleo Linux. Al interconectar estos contenedores por medio de una red virtual de tipo puente (*bridge*), se emula con precisión el comportamiento de un segmento de red física local (LAN). Esto facilita el aislamiento estricto del tráfico generado por el demonio del servidor web y las llamadas del cliente, permitiendo que un sniffer de red examine exclusivamente las unidades de datos del protocolo (PDU) deseadas.

Para el análisis del protocolo HTTP se seleccionó la combinación de Nginx como servidor web y GNU Wget como cliente HTTP. Nginx actúa como proveedor del servicio web, respondiendo solicitudes realizadas por los clientes mediante el protocolo HTTP. Por su parte, Wget permite generar solicitudes HTTP desde línea de comandos de manera simple y controlada. La elección de esta dupla cliente-servidor facilita la observación y análisis del tráfico intercambiado, permitiendo identificar claramente las solicitudes realizadas por el cliente y las respuestas generadas por el servidor.

Tráfico Esperado

Antes de realizar las capturas se esperaba observar el establecimiento de una conexión TCP entre cliente y servidor mediante el proceso conocido como *three-way handshake*. Una vez establecida la conexión, se esperaba visualizar una solicitud HTTP utilizando el método **GET** enviada por el cliente Wget hacia el servidor Nginx.

Posteriormente, el servidor debía responder con un mensaje HTTP **200 OK** acompañado por las cabeceras correspondientes y el contenido de la página web por defecto de Nginx. Finalmente, se esperaba observar el cierre de la conexión TCP mediante los segmentos de terminación correspondientes.

Debido a que ambos contenedores se encuentran conectados a una red virtual tipo *bridge* administrada por Docker, todo el tráfico generado debía permanecer dentro de dicha red virtual, facilitando su captura y análisis mediante Wireshark.

Procedimientos Realizados y Configuración de Software

El entorno experimental se construyó sobre un sistema anfitrión con distribución Arch Linux. Para el levantamiento del entorno y garantizar el control total sobre las dependencias de los contenedores, se optó por la creación de imágenes propias mediante archivos **Dockerfile**, utilizando Ubuntu como sistema operativo base.

Servidor

El servicio servidor se construyó sobre el demonio Nginx. A continuación, el **Dockerfile** utilizado para configurar la imagen del servidor:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y nginx
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Cliente

El servicio cliente se configuró instalando la herramienta GNU Wget. El contenedor se mantiene en ejecución en segundo plano para permitir la invocación interactiva de comandos desde la terminal del anfitrión:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y wget
CMD ["tail", "-f", "/dev/null"]
```

Conexión Cliente-Servidor

Para establecer la infraestructura de red aislada y ejecutar las imágenes construidas, se ejecutaron los siguientes comandos secuenciales en la terminal. Estos permiten crear la red virtual, construir las imágenes y levantar los contenedores:

```
# 1. Creacion de una red virtual en Docker
docker network create red_tarea2

# 2. Construcción de imagenes
docker build -t ubuntu-nginx ./servidor
docker build -t ubuntu-wget ./cliente

# 3. Levantamiento de contenedores conectados a la red
docker run -d --name servidor_http --network red_tarea2 ubuntu-nginx
docker run -d --name cliente_http --network red_tarea2 ubuntu-wget

# 4. Verificación de contenedores activos
docker ps
```

Como se observa en los anexos, los contenedores fueron desplegados correctamente y permanecieron activos durante toda la ejecución del experimento, permitiendo establecer la comunicación cliente-servidor necesaria para el análisis del protocolo HTTP.

Posteriormente, se procedió a identificar la interfaz de red virtual autogenerada por el controlador *bridge* de Docker mediante la consulta de enlaces de red del *kernel* (**ip link**). Localizado el identificador exacto de la interfaz (**br-2ed43d614f56**), se ejecutó el analizador Wireshark aplicando de forma estricta un filtro de visualización **http** para aislar los paquetes de la capa de aplicación. Con la captura activa, se inyectó tráfico mediante la invocación remota de una petición HTTP desde el contenedor cliente hacia el servidor web Nginx empleando la herramienta Wget, redirigiendo la salida estándar al dispositivo nulo:

```
# Inyección de solicitud de tráfico desde el cliente hacia el servidor Nginx
docker exec cliente_http wget -O /dev/null http://servidor_http
```

Análisis de Tráfico Observado

La Figura 3 muestra la captura obtenida mediante Wireshark sobre la red virtual creada por Docker. En ella puede observarse la solicitud HTTP enviada por el cliente Wget y la respuesta generada por el servidor Nginx.

La captura de datos crudos arrojada por Wireshark (cuyas evidencias gráficas se adjuntan formalmente en la sección de Anexos, en conformidad con las pautas estructurales del informe) refleja con total nitidez el ciclo de comunicación HTTP/1.1 elemental. Al aplicar el filtro de capa de aplicación, el analizador discrimina el tráfico subyacente de control de flujo de la capa de transporte (tales como los segmentos de sincronización y reconocimiento del *3-way handshake* de TCP) y expone únicamente las transacciones HTTP.

El análisis de la PDU capturada revela dos hitos operacionales críticos:

1. **Petición del Cliente (Paquete 4):** Proveniente de la dirección IP de origen **172.19.0.3** (asignada dinámicamente al contenedor **cliente_http**) con destino a la dirección IP **172.19.0.2** (**servidor_http**). Se observa el método **GET / HTTP/1.1**, acompañado por campos de cabecera característicos como **User-Agent: Wget/[versión]**, el campo obligatorio **Host: servidor_http** que permite la resolución de hosts virtuales en el servidor, y la directiva **Connection: Keep-Alive** definida por la lógica interna de Wget.
2. **Respuesta del Servidor (Paquete 6):** Emitida por la IP del servidor hacia el cliente. El mensaje de control inicia con la línea de estado **HTTP/1.1 200 OK**, ratificando que la solicitud fue procesada con éxito y que el recurso solicitado está disponible. En el desglose de cabeceras se identifica el campo **Server: nginx/[versión]**, metadatos de sincronización temporal (**Date**), el tipo MIME del objeto transferido (**Content-Type: text/html**) y la longitud exacta del cuerpo del mensaje en bytes (**Content-Length**). Inmediatamente después de la doble línea de retorno de carro y salto de línea (**\r\n**), se adjunta la carga útil (*payload*) correspondiente al código de la página web de bienvenida por defecto de Nginx.

Modificación de Tráfico y Análisis de Métricas (Suposiciones del Escenario)

En el contexto del análisis avanzado de redes, la interceptación y manipulación de paquetes sobre el canal de comunicación (**on-the-fly**) representa una técnica crítica tanto para la evaluación de la resiliencia del software como para auditorías de seguridad. Si se emplearan herramientas de enrutamiento intermedio o proxies transparentes capaces de reescribir los bits de la PDU HTTP en tránsito, las implicaciones en el comportamiento de los demonios de software serían severas y asimétricas.

- **Alteración de la Línea de Estado en la Respuesta:** Si un agente intermedio interceptara el Paquete emitido por Nginx y alterara la cadena binaria de la línea de estado, sustituyendo el código **200 OK** por un código de error de la familia de cliente como **403 Forbidden** o **404 Not Found**, la repercusión en el servidor Nginx sería nula, dado que para sus registros internos la transacción concluyó con éxito. Sin embargo, en el extremo del cliente, la lógica interna de Wget procesaría el nuevo código de estado mutado, interrumpiendo abruptamente el almacenamiento del flujo de datos en el disco duro, imprimiendo un mensaje de error crítico de acceso denegado o recurso inexistente en la consola de comandos de Arch Linux y finalizando su proceso con un código de salida distinto de cero (*non-zero exit status*).
- **Manipulación de Cabeceras de Longitud de Datos (Content-Length):** Si se alterara la cabecera **Content-Length** modificando su valor nominal a uno significativamente inferior al tamaño real del archivo HTML enviado por Nginx, el software Wget leería únicamente la cantidad de bytes especificada en el header manipulado, descartando el resto del flujo de datos e interpretando que la transacción finalizó correctamente, lo que resultaría en una descarga de archivo truncada y corrupta. Por el contrario, si el valor se modificara de forma maliciosa a una escala superior al tamaño real del recurso, Wget mantendría el socket TCP abierto en estado de escucha activa, esperando recibir los bytes faltantes hasta que se active el mecanismo de temporización por inactividad (**timeout**) del software cliente, degradando las métricas de eficiencia temporal de la red.

Conclusiones

La ejecución práctica de esta actividad permitió contrastar los conceptos teóricos asociados al protocolo HTTP con su implementación real en un entorno controlado basado en contenedores Docker. Mediante la interacción entre

un servidor Nginx y un cliente GNU Wget fue posible observar directamente el modelo de comunicación cliente-servidor característico de este protocolo y analizar las unidades de datos intercambiadas durante una transacción HTTP.

El uso de Wireshark permitió identificar los principales elementos involucrados en la comunicación, incluyendo la solicitud HTTP GET generada por el cliente y la respuesta HTTP 200 OK emitida por el servidor. Asimismo, se comprobó la estrecha relación existente entre HTTP y TCP, observando tanto el intercambio de mensajes de aplicación como el establecimiento y cierre de la conexión de transporte.

Una de las principales ventajas observadas de la contenedorización radica en su capacidad para aislar el tráfico generado por los servicios bajo estudio, permitiendo que la captura en Wireshark se enfoque exclusivamente en las comunicaciones relevantes para el experimento. Esto facilitó el análisis de los paquetes intercambiados y mejoró la reproducibilidad de las pruebas realizadas.

Por otra parte, el análisis de los posibles efectos derivados de la modificación del tráfico permitió comprender la importancia de los campos de control presentes en los mensajes HTTP. Alteraciones en códigos de estado o cabeceras como **Content-Length** pueden provocar comportamientos inesperados en los clientes, afectando la correcta recepción e interpretación de los recursos solicitados.

En definitiva, se cumplieron satisfactoriamente los objetivos planteados para esta actividad, adquiriendo experiencia práctica en el despliegue de servicios mediante contenedores, la captura y análisis de tráfico de red, y la comprensión del funcionamiento interno del protocolo HTTP en un entorno cliente-servidor.

Referencias

1. Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP Semantics (RFC 9110)*. Internet Engineering Task Force (IETF).
2. Docker Documentation. (2026). *Docker Compose CLI reference and network architecture options*. Recuperado de <https://docs.docker.com/>
3. Nginx Reference Documentation. (2026). *Nginx HTTP Core Module Architecture*. Recuperado de <https://nginx.org/en/docs/>
4. Free Software Foundation. (2025). *GNU Wget Manual*. Recuperado de <https://www.gnu.org/software/wget/manual/>
5. Wireshark Foundation. (2026). *Wireshark User Guide*. Recuperado de <https://www.wireshark.org/docs/>

Anexos

Con el fin de preservar el orden formal del documento y dar cumplimiento irrestricto a los criterios metodológicos estipulados por la Facultad de Ingeniería y Ciencias de la Universidad Diego Portales, todas las evidencias visuales, registros gráficos de instalación y capturas de flujos de datos se agrupan de forma exclusiva dentro de esta sección analítica, habiendo sido debidamente referenciados a lo largo del texto de desarrollo principal.

Evidencias de Configuración e Infraestructura

```
[pepo@Pepo-Linux tarea2]$ docker build -t ubuntu-nginx ./servidor
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:latest
--> f3d28607ddd7
Step 2/4 : RUN apt-get update && apt-get install -y nginx
--> Using cache
--> 958d76b21a7e
Step 3/4 : EXPOSE 80
--> Using cache
--> 69d9d05e8450
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
--> Using cache
--> a8dda080b7bc
Successfully built a8dda080b7bc
Successfully tagged ubuntu-nginx:latest
[pepo@Pepo-Linux tarea2]$ docker build -t ubuntu-wget ./cliente
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu:latest
--> f3d28607ddd7
Step 2/3 : RUN apt-get update && apt-get install -y wget
--> Using cache
--> 157a13e9ecfd
Step 3/3 : CMD ["tail", "-f", "/dev/null"]
--> Using cache
--> 2993ec30578a
Successfully built 2993ec30578a
Successfully tagged ubuntu-wget:latest
[pepo@Pepo-Linux tarea2]$
```

Figura 1: Captura del proceso de construcción de imágenes Docker a partir de los Dockerfile creados.

```
[pepo@Pepo-Linux tarea2]$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
81ce1b9b5e79   ubuntu-wget    "tail -f /dev/null"     27 minutes ago Up 27 minutes          cliente_http
a835a6a39375   ubuntu-nginx   "nginx -g 'daemon of..." 28 minutes ago Up 28 minutes      80/tcp          servidor_http
```

Figura 2: Ejecución de contenedores y prueba de conexión desde el cliente Wget en la terminal.

Captura Forense de Paquetes en Wireshark

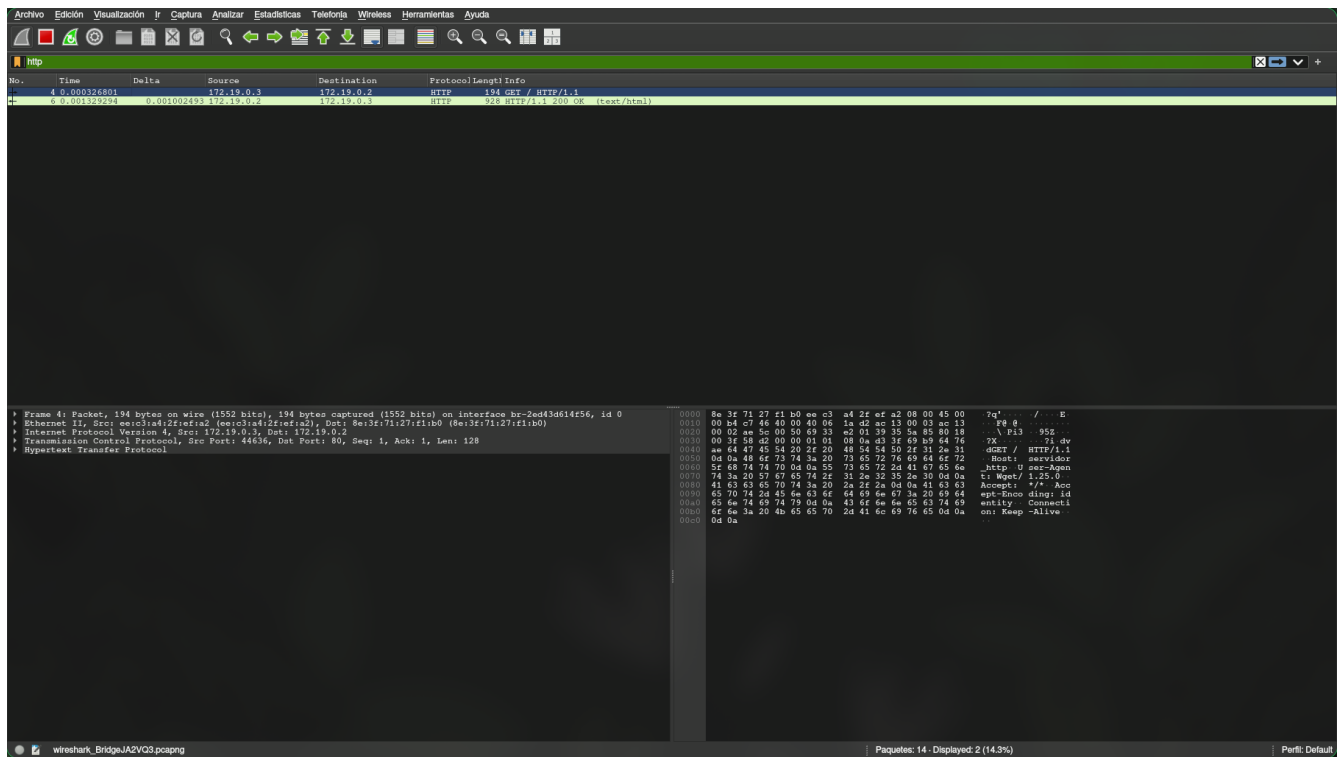


Figura 3: Captura del tráfico HTTP en Wireshark evidenciando los paquetes GET y 200 OK.